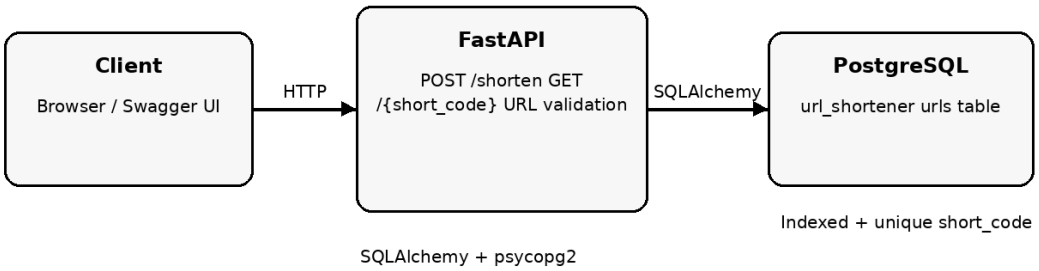
Assessment Project Documentation

URL Shortener API

The project implements a basic URL Shortener REST API using FastAPI and PostgreSQL. The API accepts a long URL, generates a unique short code, persists the mapping in PostgreSQL, and redirects users from the short code to the original URL.

TECH STACK: Python, FastAPI, PostgreSQL, Uvicorn, SQLAlchemy, psycopg2, Pydantic

ARCHITECTURE: The current implementation uses a simple three-layer flow: the client sends an HTTP request to FastAPI, FastAPI performs validation and application logic, and SQLAlchemy persists or retrieves the



URL mapping from PostgreSQL.

PROJECT FOLDER STRUCTURE:

```
url-shortener/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   └── schemas.py
├── .env
├── .env.example
├── .gitignore
└── requirements.txt
```

API DESIGN AND IMPLEMENTATION

1. POST /shorten
Accepts a long URL, validates it using Pydantic HttpUrl, generates a random six-character alphanumeric short code, checks for collisions, and stores the mapping in PostgreSQL.

Example request:

```
{
  "url": "https://www.example.com/some/long/path"
}
```

Example response:

```
{
  "short_url": "http://localhost:8000/aB7kP2"
}
```

2. GET /{short_code}
Looks up the supplied short code in PostgreSQL. If a matching record exists, the API returns a 307 Temporary Redirect response to the original URL. If the code does not exist, the API returns HTTP 404.

3. Input Validation and Error Handling
- Invalid URL input is rejected by Pydantic validation.
- Unknown short codes return HTTP 404.
- The short_code field is unique in PostgreSQL to prevent duplicate mappings.
- Database credentials are stored outside source code using environment configuration.

DATABASE DESIGN:
The implementation uses a single urls table:

Column	**Type**	**Constraint**	**Purpose**
id	Integer	Primary key	Unique record identifier
short_code	VARCHAR(10)	Unique + indexed + not null	Short URL identifier
original_url	TEXT	Not null	Destination URL
created_at	Timestamp	Server default	Creation time

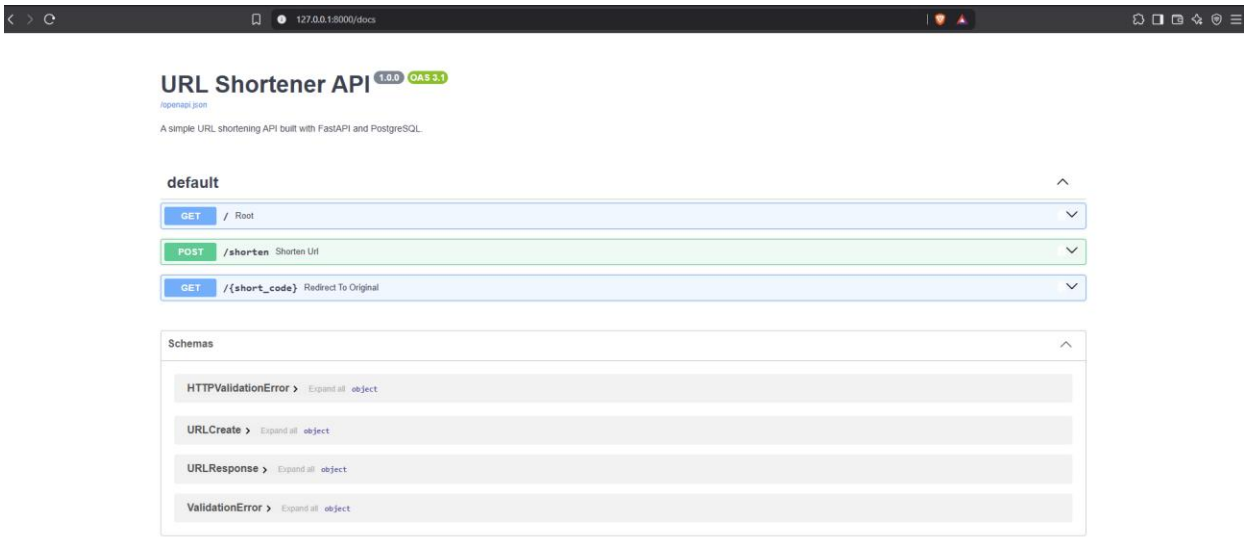
TESTING PERFORMED:
- Started the FastAPI application using Uvicorn.
- Opened FastAPI Swagger UI at /docs.
- Created a shortened URL using POST /shorten.
- Verified that the generated mapping was persisted in PostgreSQL using SELECT * FROM urls.
- Verified the short URL redirects to the original destination.
- Tested a nonexistent short code and confirmed HTTP 404.
- Tested invalid URL input and confirmed validation failure.

To run the current implementation follow below steps:

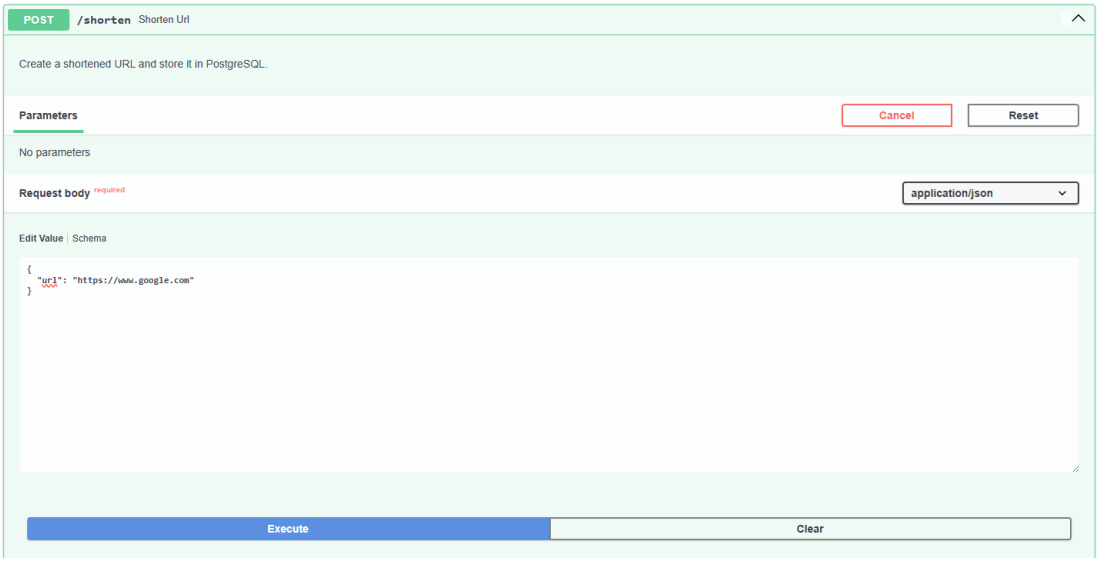
a) Install Python and PostgreSQL.
b) Create the PostgreSQL database named url_shortener.
c) Create and activate the Python virtual environment.
d) Install dependencies from requirements.txt.
e) Create .env using .env.example and provide the local PostgreSQL password.
f) Start the API with: uvicorn app.main:app --reload
g) Open Swagger UI at: http://127.0.0.1:8000/docs

Screenshots

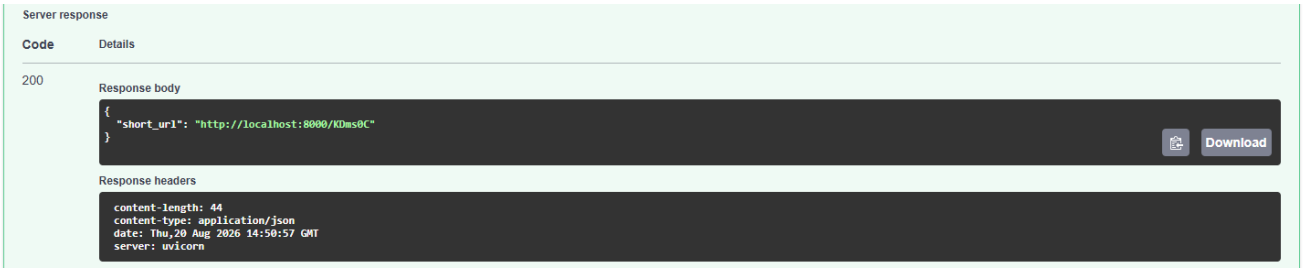
FastAPI Swagger UI showing the available endpoints.



POST /shorten request and successful response. GET /{short_code} redirect result



GET /{short_code} redirect result.



PostgreSQL SELECT query showing the same stored URL mapping.

```
psql (18.6)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

url_shortener=# SELECT * FROM urls;
 id | short_code |      original_url       |           created_at
----+------------+-------------------------+--------------------------------
  1 | KDms0C     | https://www.google.com/ | 2026-08-20 20:20:57.644563+05:30
(1 row)


url_shortener=#
```

Planned High-Level Design

The current implementation is intentionally simple for the assessment. A production URL shortener would typically become read-intensive because every shortened URL can be accessed many more times than it is created. The following improvements describe how the current design could evolve without changing the core API contract.

1 Read-Intensive Workload and Caching

The GET /{short_code} endpoint is expected to receive substantially more traffic than POST /shorten. To reduce repeated database reads, Redis can be introduced as a cache in front of PostgreSQL.

1. Check Redis for the short_code first.
2. On a cache hit, return the redirect immediately.
3. On a cache miss, query PostgreSQL.
4. Store the retrieved mapping in Redis with an appropriate TTL.
5. Continue using PostgreSQL as the persistent source of truth.

This improves latency and reduces database load while keeping PostgreSQL responsible for durable storage.

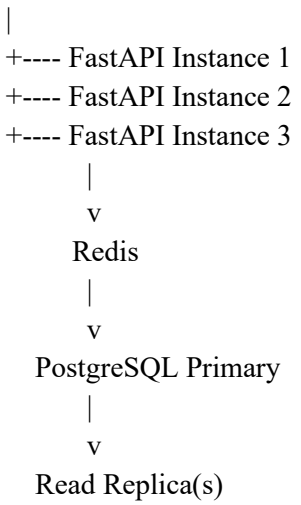
2 Low Latency

- Keep short_code indexed and unique for efficient lookup.
- Use Redis for frequently accessed mappings.
- Use database connection pooling to avoid repeatedly establishing database connections.
- Keep the redirect path lightweight and avoid unnecessary processing before issuing the redirect.

3 High Availability and Horizontal Scaling

The current application has a single FastAPI process and a single PostgreSQL instance, so it is not highly available by itself. For production, multiple stateless FastAPI instances can be placed behind a load balancer. PostgreSQL can be configured with replication and automated failover.

Client
|
v
Load Balancer



Because the API instances do not need local session state, they can scale horizontally. If one API instance fails, the load balancer can route requests to healthy instances.

4 Security

- Validate submitted URLs before storing them.
- Keep database credentials in environment variables and never commit .env.
- Use parameterized ORM/database operations to reduce SQL injection risk.
- Use HTTPS in a deployed environment.
- Introduce rate limiting to reduce API abuse and automated URL creation.
- Consider authentication/authorization if the service later supports user-specific URL management.
- Consider URL reputation or malicious-domain checks because a shortener can be abused to conceal phishing or malicious destinations.
- Add logging, monitoring, and alerting for suspicious traffic and service failures.

Concern	Current Implementation	Planned Enhancement
Read-heavy traffic	Indexed PostgreSQL lookup	Redis caching
Latency	FastAPI + indexed DB query	Redis + connection pooling
Availability	Single API process / DB	Load balancer + multiple API instances + DB failover
Database scalability	Single PostgreSQL database	Replication / read replicas
Security	URL validation + environment secrets	HTTPS + rate limiting + WAF/API gateway + abuse detection
Observability	Basic application output	Centralized logs, metrics, tracing, alerts